

SEED Labs — Blockchain Exploration Lab 实验报告

徐锦云 23307130447 信息安全

SEED Labs — Blockchain Exploration Lab 实验报告

1. 实验概述
2. 实验环境搭建
 - 2.1 启动模拟器
 - 2.2 节点列表 (部分)
 - 2.3 网络拓扑 (简化)
 - 2.4 预置账户 (助记词恢复)
3. Task 1: MetaMask 钱包配置
 - 3.1 安装与连接 (Task 1.a & 1.b)
 - task 1.a
 - task 1.b
 - 3.2 恢复账户 (Task 1.c)
 - 3.3 发送交易 (Task 1.d)
4. Task 2: Python 与区块链交互
 - 4.1 安装依赖 (Task 2.a)
 - 4.2 查询余额 (Task 2.b)
 - 4.3 发送交易 (Task 2.c)
5. Task 3: Geth 控制台交互
 - 5.0 如何找到并进入以太坊节点容器
 - 5.1 查询余额 (Task 3.a)
 - 方法一: 不进入容器, 宿主机一条命令执行 (推荐)
 - 方法二: 进入容器后交互执行
 - 5.2 从节点本地账户发送 (Task 3.b)
 - 5.3 从 MetaMask 账户发送 (Task 3.c)
6. Task 4: 添加 Full Node
 - 6.1 配置流程
 - 6.2 验证 Peers
 - 6.3 创建 Account Z
 - 6.4 经新节点发送交易 (Python)
 - Step 1 — 确认新节点 RPC 可用
 - Step 2 — 使用 Task 4 专用脚本 (推荐)
 - Step 3 — 运行脚本
 - 6.5 从 Account Z 发出交易 (Geth 控制台)
 - Step 1 — 进入新节点 Geth 控制台
 - Step 2 — 确认 Account Z 是本地第一个账户
 - Step 3 — 查看 Account Z 当前余额 (应有 6.4 转入的 5 ETH)
 - Step 4 — 解锁并发交易
 - Step 5 — 验证余额变化
 - 6.6 加入网络流程图
7. 问题回答与观察
 - Q1: 为什么实验使用 PoA 而不是 PoS?
 - Q2: 助记词 (Mnemonic) 的作用是什么?
 - Q3: 为什么 Geth 控制台无法从 MetaMask 账户发交易?
 - Q4: 连接不同节点 RPC 发送交易有区别吗?
 - Q5: PoA 网络中 bootnode 的作用?
 - Q6: gas 费用如何影响余额?

1. 实验概述

本实验在 SEED Internet Emulator 中搭建一条 **Proof-of-Authority (PoA)** 私有以太坊链，通过四种方式与链交互：

任务	交互方式	核心目标
Task 1	MetaMask 浏览器钱包	连接节点、恢复账户、发送交易
Task 2	Python + web3.py	查询余额、构造并签名原始交易
Task 3	Geth JavaScript 控制台	节点本地账户操作、IPC 交互
Task 4	新建 Full Node	加入现有网络、创建账户、跨节点转账

共识机制说明： 本实验链使用 PoA (Clique) ，而非以太坊主网的 PoS。PPT 中强调：PoA 由预授权签名者轮流出块，适合教学仿真，不依赖质押或算力竞争。

2. 实验环境搭建

2.1 启动模拟器

在 `emulator_20` 目录下执行:

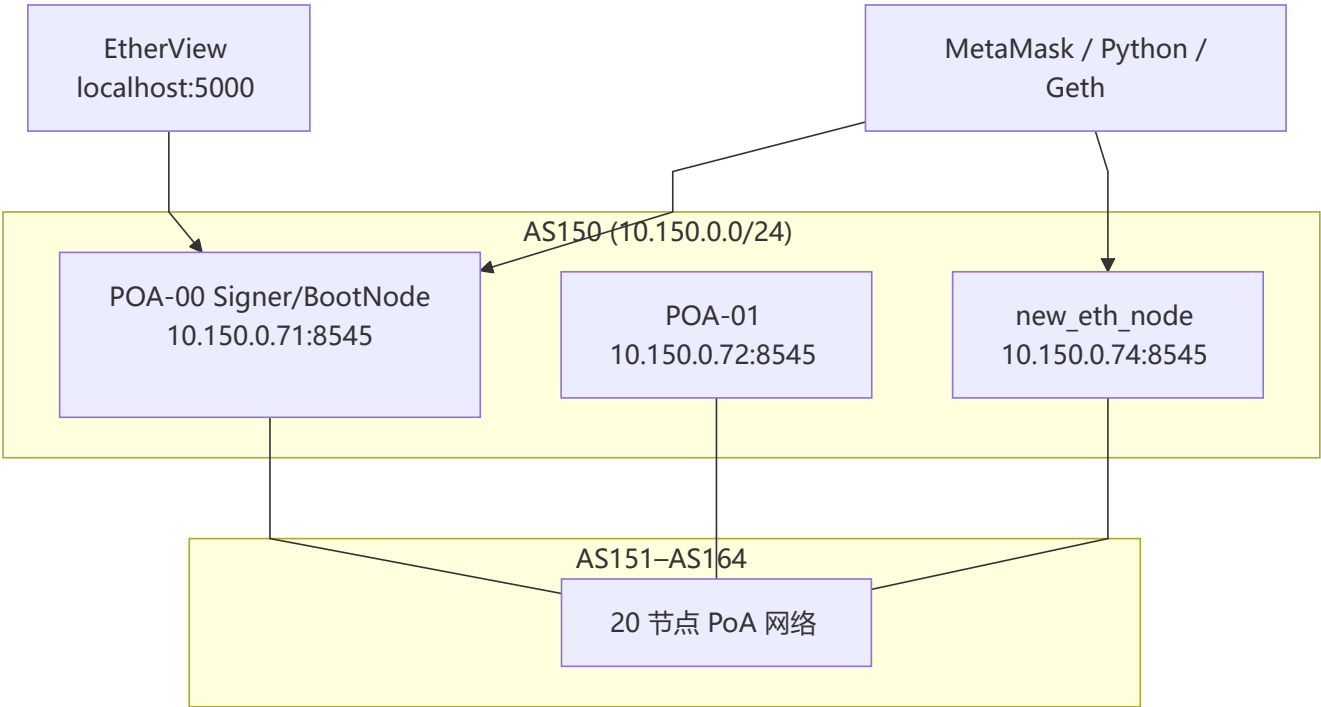
```
cd /home/seed/Desktop/Labsetup/emulator_20
dcbuild      # docker-compose build
dcup         # docker-compose up
```

启动后共运行 20 个以太坊节点 + 1 个 EtherView 监控服务 + 1 个空白 new eth node 容器。

2.2 节点列表 (部分)

[illegible]

2.3 网络拓扑（简化）



2.4 预置账户（助记词恢复）

模拟器在创世块中预置了 3 个用户账户，由以下助记词派生：

```
gentle always fun glass foster produce north tail security list example gain
```

账户配置见 `emulator_code/blockchain-poa.py`：

```
# These 3 accounts are generated from the following phrase:
# "gentle always fun glass foster produce north tail security list example gain"
# They are for users. We will use them in MetaMask, as well as in our sample code.
blockchain.addLocalAccount(address='0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9',
                             balance=30)
blockchain.addLocalAccount(address='0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9',
                             balance=9999999)
blockchain.addLocalAccount(address='0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24',
                             balance=10)
```

序号	地址	初始余额
Account 1	0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9	30 ETH
Account 2	0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9	9,999,999 ETH
Account 3	0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24	10 ETH

3. Task 1: MetaMask 钱包配置

3.1 安装与连接 (Task 1.a & 1.b)

步骤:

- 1. 在 Firefox 中搜索并安装 MetaMask 扩展 (开发者: danfinlay) 。
- 2. 进入 **Settings** → **Networks** → **Add a network manually**, 填写:

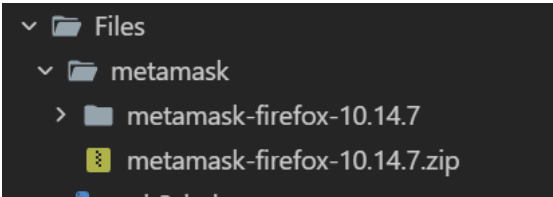
字段	值
Network name	SEED emulator
New RPC URL	http://10.150.0.71:8545
Chain ID	1337
Currency symbol	ETH

- 3. 连接成功后, MetaMask 应显示当前网络为自定义链, Chain ID = 1337。

task 1.a

下载与 Firefox 83 兼容的 MetaMask 版本:

```
$ mkdir -p /home/seed/Desktop/Labsetup/Files/metamask && cd /home/seed/Desktop/Labsetup/Files/metamask && curl -sL -o metamask-firefox-10.14.7.zip "https://github.com/MetaMask/metamask-extension/releases/download/v10.14.7/metamask-firefox-10.14.7.zip" && unzip -o metamask-firefox-10.14.7.zip -d metamask-firefox-10.14.7 && ls -la metamask-firefox-10.14.7/ | head -5
```

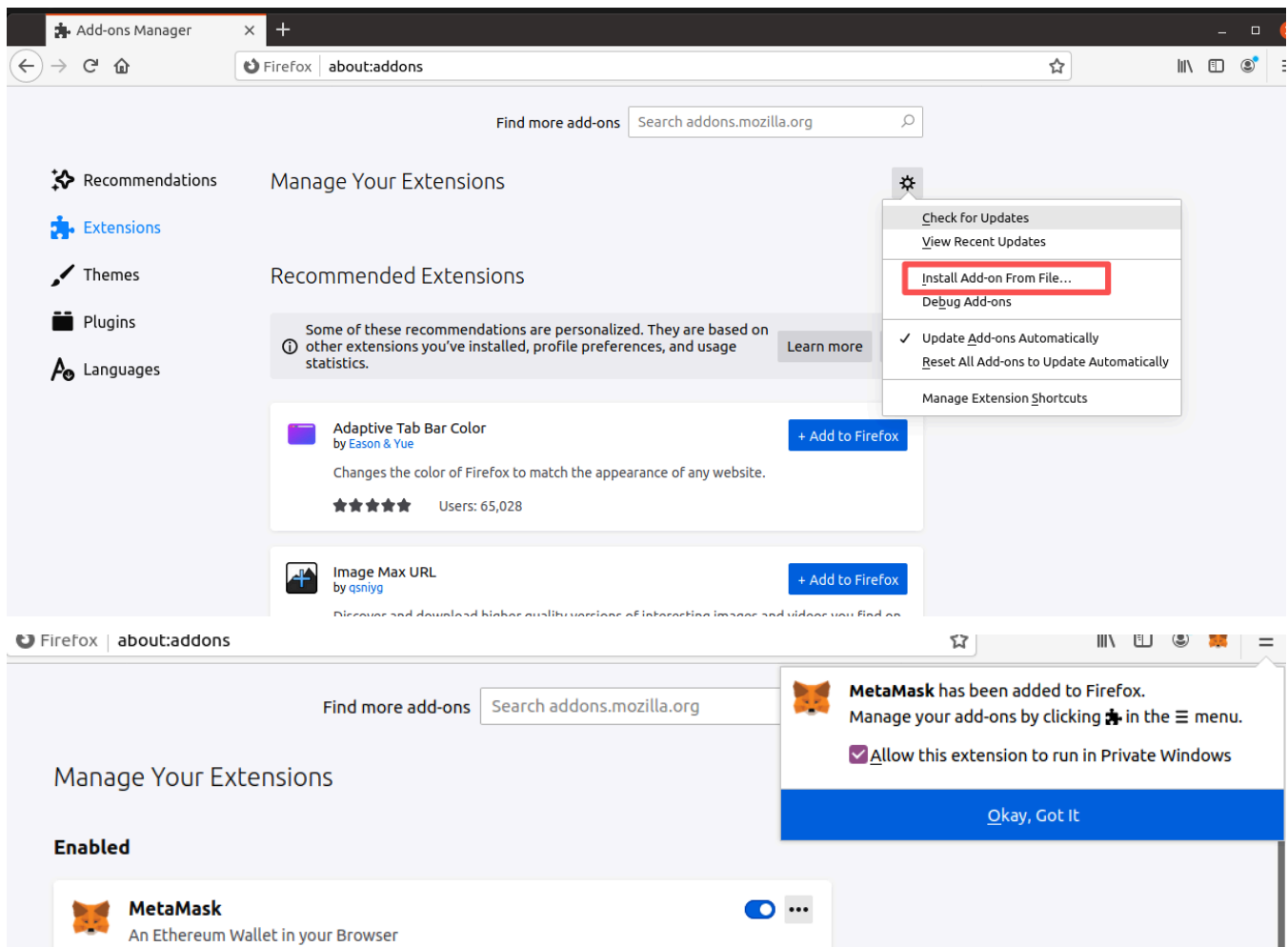


- 1. 地址栏输入 `about:config` 回车
- 2. 点 "Accept the Risk and Continue"
- 3. 搜索: `xpinstall.signatures.required`
- 4. 双击把它改成 `false`



- 5. 回到 `about:addons` → 齿轮 ⚙ → Install Add-on From File
- 6. 再选 `metamask-firefox-10.14.7` 文件夹里的文件安装
- 7. 点 "添加" 完成安装

装好后右上角会出现 MetaMask 狐狸图标.



task 1.b

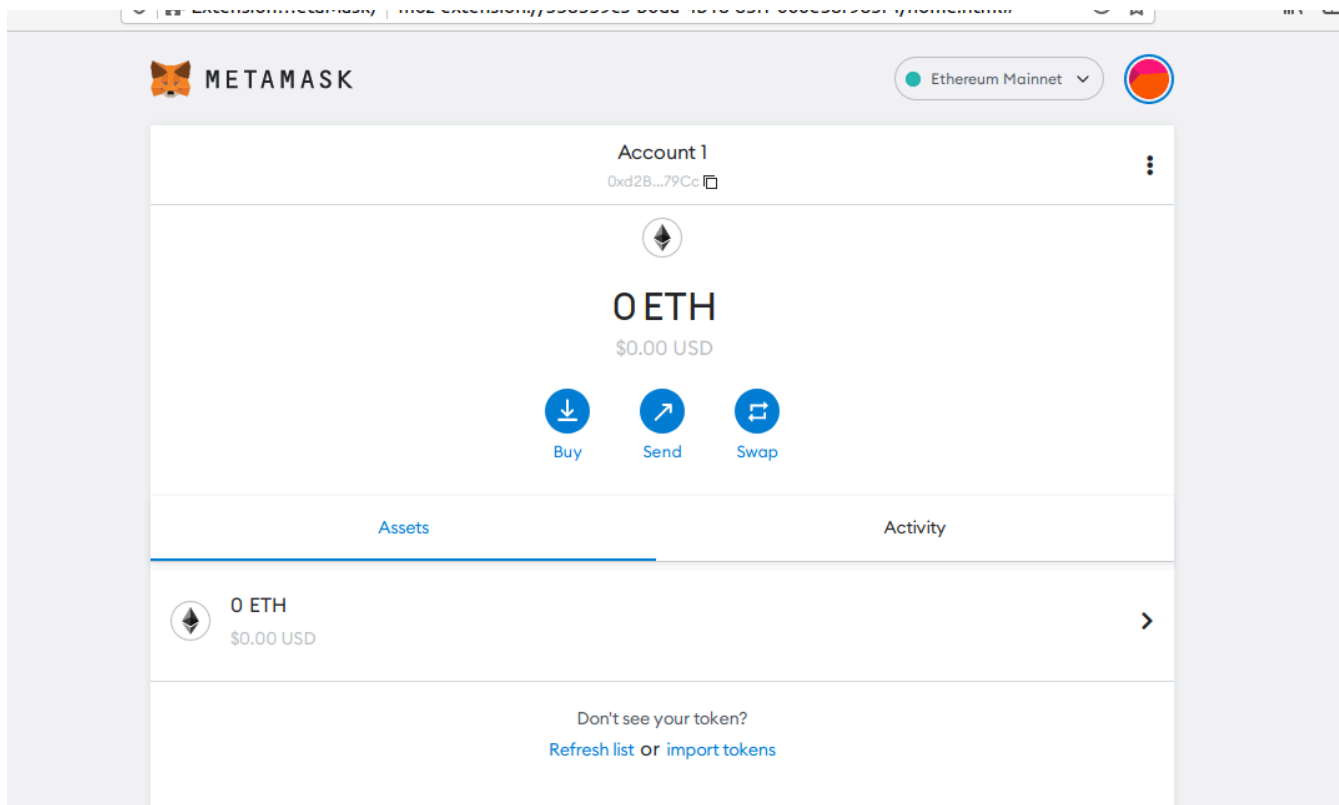
第一步：先打开 MetaMask：点 Firefox 右上角的 狐狸图标，确保 MetaMask 窗口已弹出（若首次使用，先点 “Get Started” 创建/导入钱包，后面 Task 1.c 再用助记词也行）。我设置的密码是：seedseed

Secret Recovery Phrase

Your Secret Recovery Phrase makes it easy to back up and restore your account.

WARNING: Never disclose your Secret Recovery Phrase. Anyone with this phrase can take your Ether forever.

piece winter element smart nominee
behave act electric have blur
memory name



到这里就是账户创建好了，现在开始实验设置。

第二步：找到“添加网络”入口：在 MetaMask 窗口最上方，当前网络名称（默认可能是 Ethereum Mainnet）处点击，会弹出网络列表。在列表最底部找下面其中一个（不同小版本文字略有差异）：

- Custom RPC
- Add Network
- Custom Network

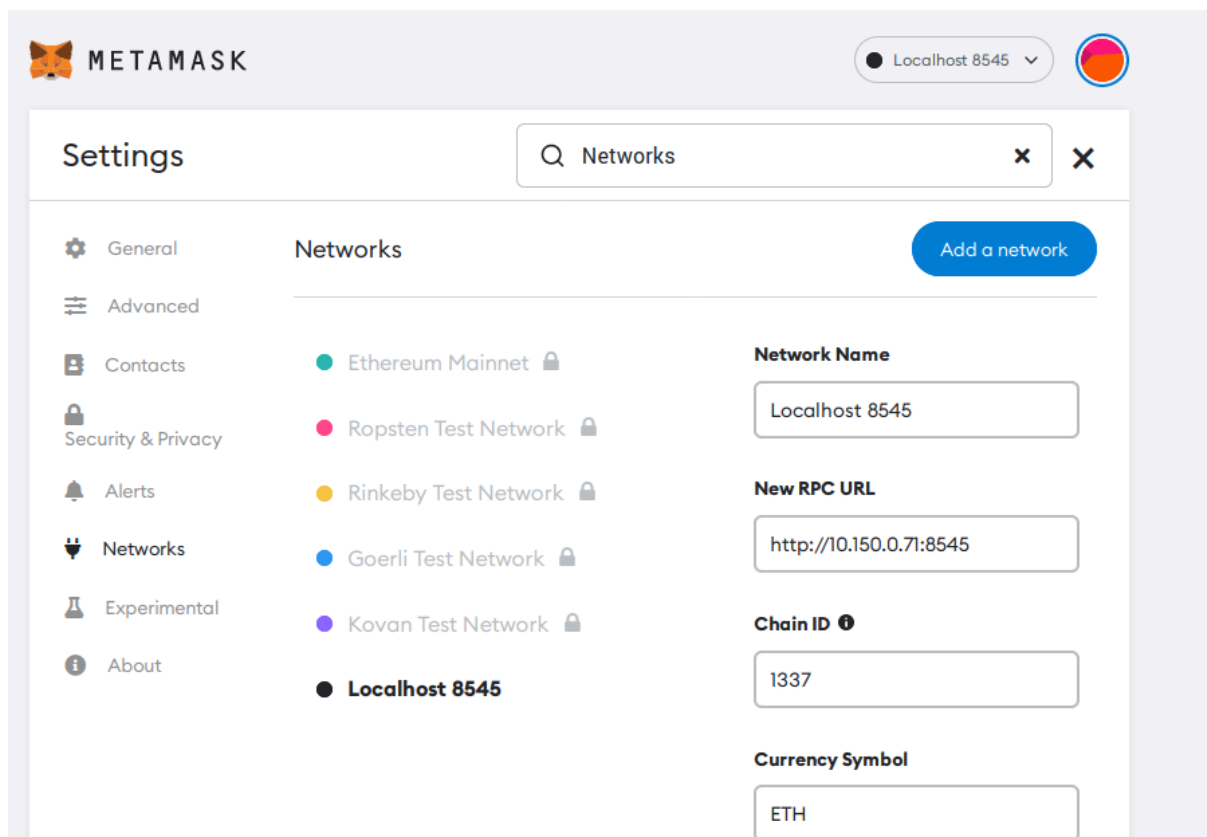
点它，进入手动填 RPC 的页面。

第三步：填写网络信息：按实验要求填入（IP 用下面这个即可）：

字段	填什么
Network Name（网络名称）	Localhost 8545
New RPC URL / RPC URL	http://10.150.0.71:8545
Chain ID（链 ID）	1337
Symbol / Currency Symbol	ETH
Block Explorer URL（区块浏览器）	留空

然后点 Save / Add / Save Network。

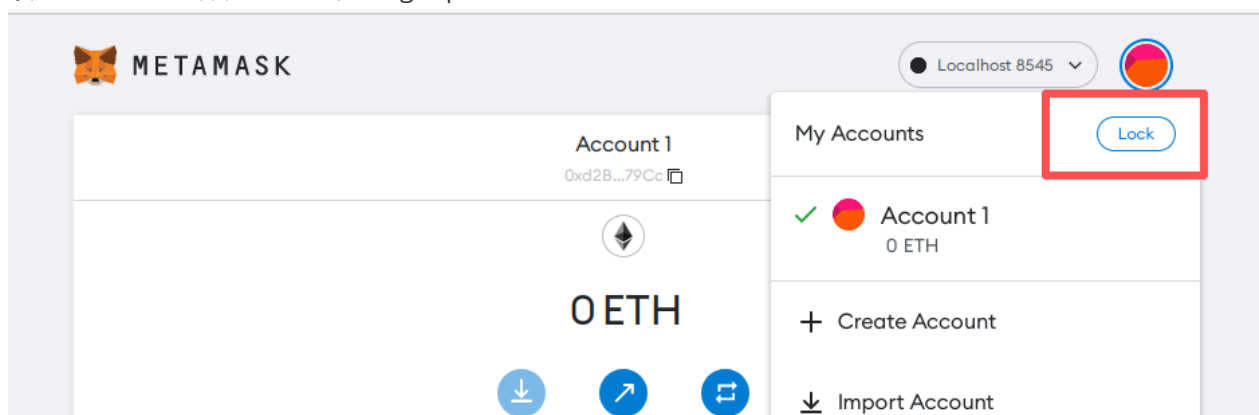
第四步：切换到该网络：保存后，再点顶部网络名称，选中 Localhost 8545。顶部应显示刚建的网络。



3.2 恢复账户 (Task 1.c)

操作流程:

1. 锁定 MetaMask 账户 → 点击 "Forgot password"



2. 输入助记词: gentle always fun glass foster produce north tail security list example gain; 我重新设置的密码是 deesdees
3. MetaMask 通过 BIP-39/BIP-44 路径 `m/44'/60'/0'/0/{index}` 派生账户, 并显示链上有余额的账户

算法分析 — HD 钱包派生:

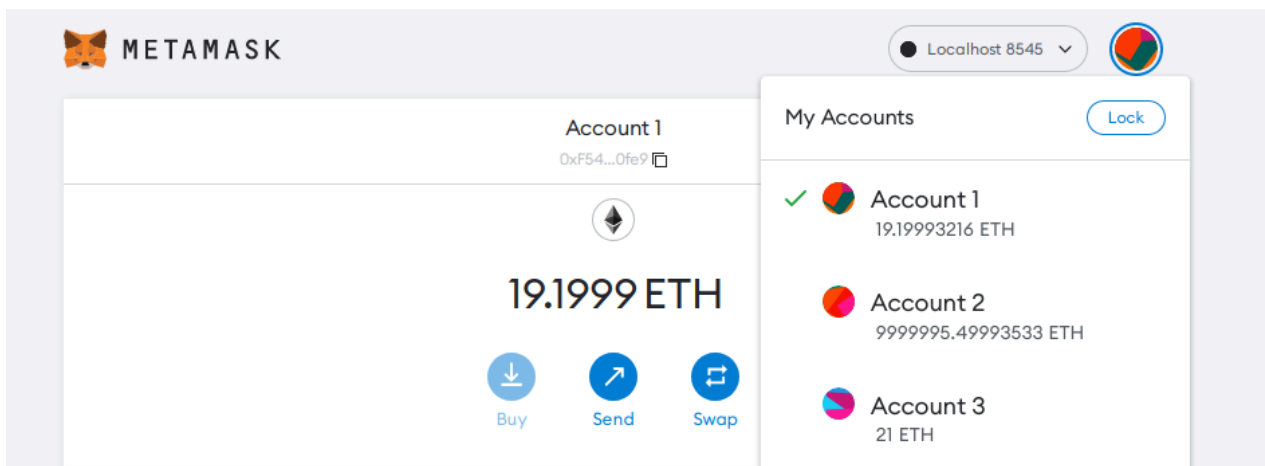
助记词 (BIP-39)
↓ PBKDF2 生成 512-bit 种子
种子
↓ BIP-32 主密钥
↓ BIP-44 路径 `m/44'/60'/0'/0/i`
私钥 → 公钥 → Keccak-256 → 以太坊地址 (20 bytes)

MetaMask 使用此标准从同一助记词确定性恢复所有账户，无需单独备份每个私钥。

Python 验证派生结果：

```
from eth_account import Account
Account.enable_unaudited_hdwallet_features()
acct = Account.from_mnemonic(
    "gentle always fun glass foster produce north tail security list example gain",
    account_path="m/44'/60'/0'/0/0"
)
# acct.address = 0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9
```

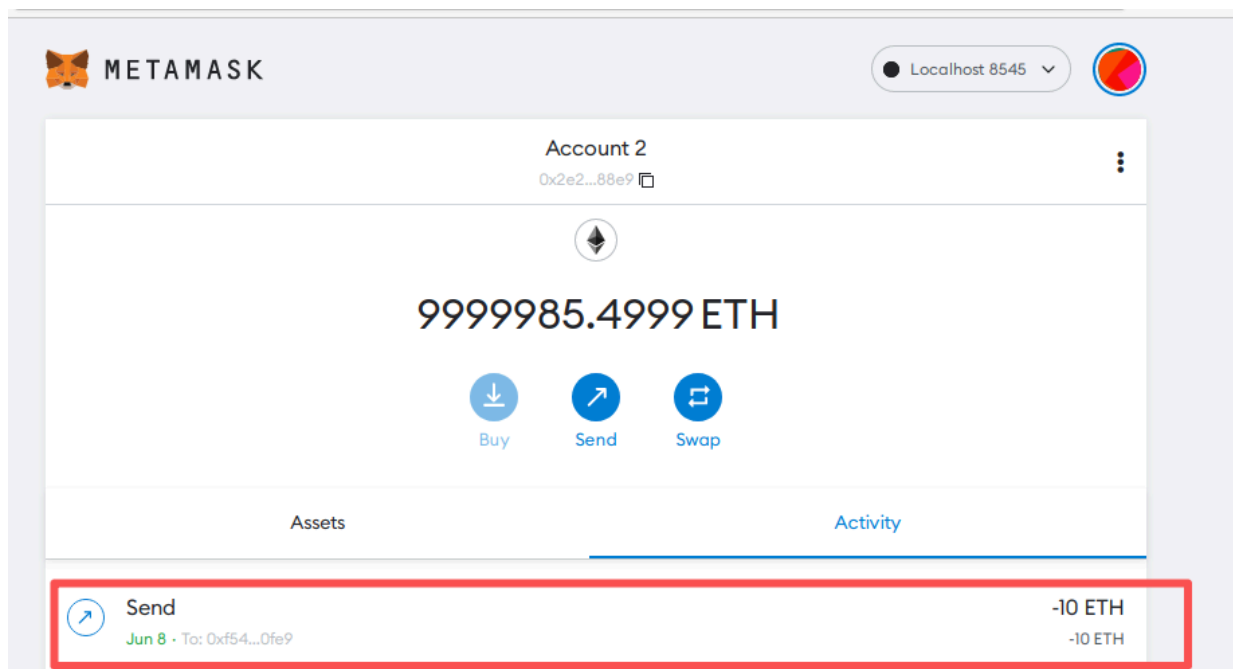
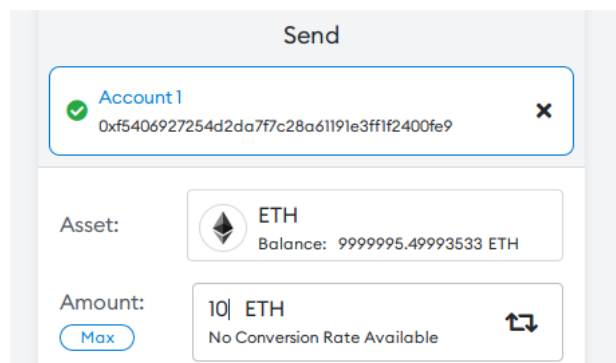
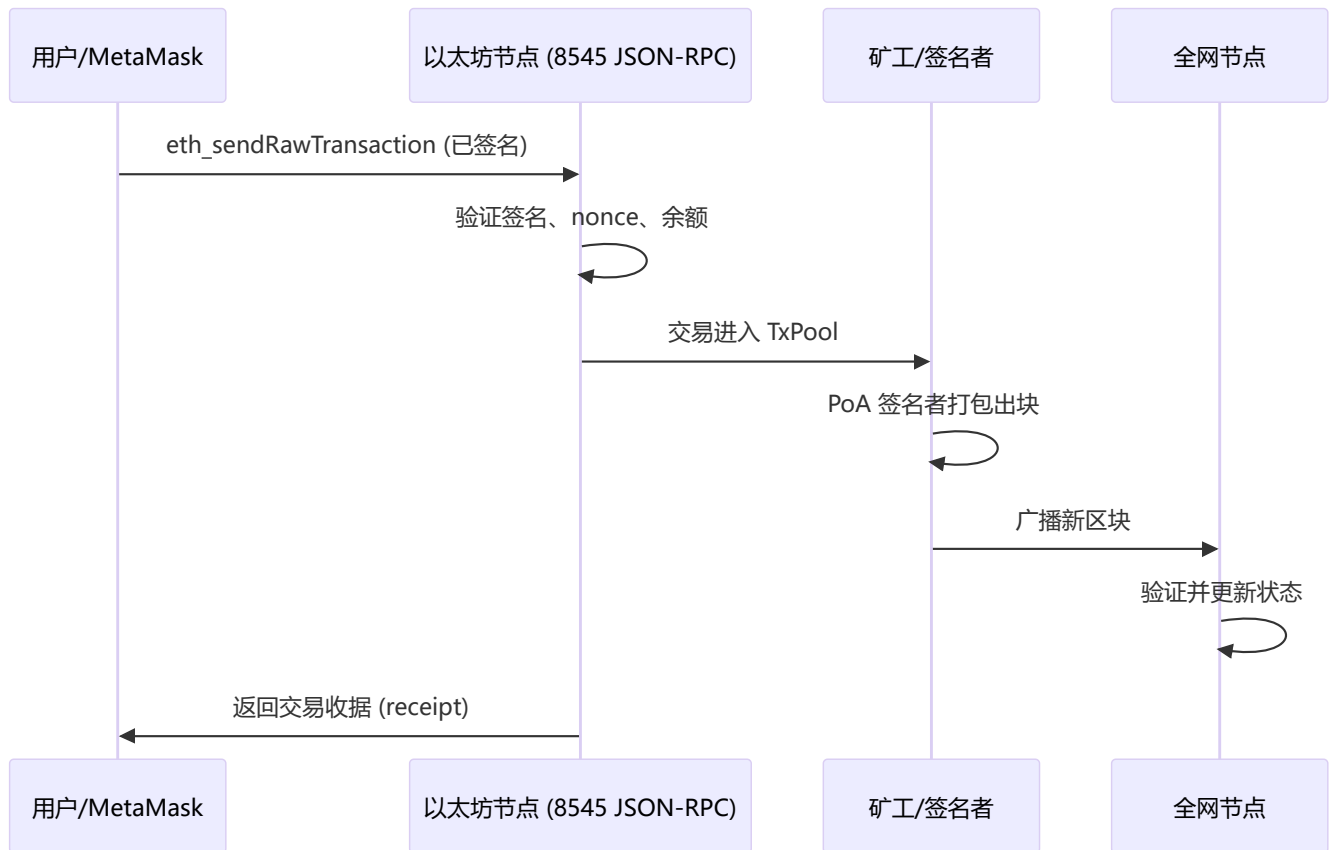
这是后续补图，三个账号：



3.3 发送交易 (Task 1.d)

在 MetaMask 中从 Account 2 向 Account 1 发送任意金额（如 1 ETH），然后在 EtherView（<http://localhost:5000/block-view/>）查看区块与交易详情，核对 from/to/value 是否一致。

交易上链流程：



4. Task 2: Python 与区块链交互

4.1 安装依赖 (Task 2.a)

```
pip3 install web3==5.31.1 docker
```

4.2 查询余额 (Task 2.b)

原始代码位于 `Files/web3_balance.py`:

```
#!/bin/env python3
from web3 import Web3

url = 'http://10.150.0.71:8545'
web3 = Web3(Web3.HTTPProvider(url)) # Connect to a blockchain node

addr = web3.toChecksumAddress('0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9')
balance = web3.eth.get_balance(addr) # Get the balance
print(addr + ": " + str(web3.fromwei(balance, 'ether')) + " ETH")
```

实验结果 (实验开始时):

账户	Python 查询余额	与预置一致
Account 0	30 ETH	✓
Account 1	9,999,999 ETH	✓
Account 2	10 ETH	✓

原理: `web3.eth.get_balance()` 通过 JSON-RPC 调用 `eth_getBalance`, 节点返回该地址在最新状态下的 Wei 余额。

以下是account 1余额查询:

```
● [06/08/26]seed@VM:~/.../Labsetup$ python3 Files/web3_balance.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3
(2.2.3) or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}) or chardet ({}) doesn't match a supported "
0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9: 29.199932159851183 ETH
```

4.3 发送交易 (Task 2.c)

基于 `Files/web3_raw_tx.py` 完成, 关键配置:

```
web3 = Web3(Web3.HTTPProvider('http://10.150.0.71:8545'))
key = '0x72c28c0d980b5e26435fc7eb8afaa27a5a117359669d73284f69ed8a401c6a85' # Account 0 私钥
sender = Account.from_key(key)
recipient = web3.toChecksumAddress('0xCBf1e330F0abb5c1ac979CF2B2B874cfD4902E24') # Account
2
```

```

4 web3 = Web3(Web3.HTTPProvider('http://ip-address:8545'))
5 web3 = Web3(Web3.HTTPProvider('http://10.150.0.71:8545'))
6
7 # Sender's private key
8 key = 'private key'
9 # Sender's private key (Account 0)
10 key = '0x72c28c0d980b5e26435fc7eb8afaa27a5a117359669d73284f69ed8a401c6a85'
11 sender = Account.from_key(key)
12
13 recipient = Web3.toChecksumAddress('account number')
14 recipient = Web3.toChecksumAddress('0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24')

```

交易参数分析：

字段	值	含义
chainId	1337	防止跨链重放攻击
nonce	动态获取	该发送者地址已发出交易数，防重放
value	某个 ETH	转账金额
gas	200000	Gas 上限
maxFeePerGas	4 gwei	EIP-1559 最高 gas 费
maxPriorityFeePerGas	3 gwei	给矿工的小费

执行结果：

```

[06/08/26]seed@VM:~/.../Labsetup$ python3 Files/web3_raw_tx.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (2.2.3)
or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}) or chardet ({}) doesn't match a supported "
Transaction sent, waiting for receipt ...

===== Transaction Receipt =====
Status:      Success
Tx Hash:     0xe6ba582c8327fbdc69f3a5306dbc06538749cc3e4d4697c93421412c3c634860
Block Number: 353
From:        0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9
To:          0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24
Gas Used:    21000
=====

Sender balance: 7.199806159850889 ETH
Recipient balance: 53 ETH

```

签名与发送流程：

1. 构造交易字典 tx
2. RLP 编码 + ECDSA(secp256k1) 签名 → signed_tx
3. eth_sendRawTransaction 广播到节点
4. wait_for_transaction_receipt 轮询直到打包进块

与 MetaMask 的区别：MetaMask 在浏览器内完成签名；Python 程序在本地用 `eth_account` 库离线签名，再通过 RPC 发送 raw transaction。

5. Task 3: Geth 控制台交互

5.0 如何找到并进入以太坊节点容器

Step 1 — 查看正在运行的容器：

```
# SEED VM 可用别名
dockps

# 或通用命令
docker ps --format "{{.ID}} {{.Names}}"
```

在输出中找到名称含 `Ethereum-POA` 的行，例如：

```
[06/08/26]seed@VM:~/.../emulator_20$ dockps
d20af4eda58c  as154h-Ethereum-POA-08-Signer-10.154.0.71
b0195146937a  as161h-Ethereum-POA-12-Signer-BootNode-10.161.0.71
52551bc8b52b  as150h-new_eth_node-10.150.0.74
14563deaa097  as160h-Ethereum-POA-11-10.160.0.72
f95d17c6cf02  as150h-Ethereum-POA-01-10.150.0.72
c275001d11c6  as161h-Ethereum-POA-13-10.161.0.72
ae3cb96a4ce7  as164h-Ethereum-POA-18-Signer-BootNode-10.164.0.71
ad1c6bebc36a  as160h-Ethereum-POA-10-Signer-10.160.0.71
f3a40a8add0c  as152h-Ethereum-POA-04-Signer-10.152.0.71
98cc7dda5c2b  as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71
3b8ff8726157  as151h-Ethereum-POA-02-Signer-10.151.0.71
3170671aad99  as162h-Ethereum-POA-15-BootNode-10.162.0.72
7a8ce5787a97  as153h-Ethereum-POA-06-Signer-BootNode-10.153.0.71
ead6b30eb3ee  as164h-Ethereum-POA-19-10.164.0.72
26a1a8eb5f5e  as154h-Ethereum-POA-09-BootNode-10.154.0.72
9da2d4d88806  as153h-Ethereum-POA-07-10.153.0.72
```

末尾 `10.150.0.71` 即节点 IP，Task 3 任选其一即可，本实验使用第一个 BootNode。

Step 2 — 进入容器（两种方式任选）：

方式	命令	说明
A. 快捷别名	<code>docksh 98c</code>	<code>98c</code> 为容器 ID 前几位，须唯一
B. 完整命令	<code>docker exec -it as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 /bin/bash</code>	用完整容器名

进入后提示符变为 `root@<容器ID>:/#`，表示已在容器内。

```
/06a0d5e23f3 seedemu_internet_map
[06/08/26]seed@VM:~/.../emulator_20$ docksh 98c
root@98cc7dda5c2b / #
```

Step 3 — 打开 Geth 交互控制台:

```
geth attach
```

出现 `welcome to the Geth JavaScript console!` 和 `>` 提示符即可输入 JavaScript 命令。退出按 `Ctrl+D`。

```
root@98cc7dda5c2b / # geth attach
Welcome to the Geth JavaScript console!

instance: Geth/NODE_1/v1.10.26-stable-e5eb32ac/linux-amd64/go1.18.10
coinbase: 0x1be9288f9a7d2f809250f15c487e3a5a9cf71f4f
at block: 384 (Mon Jun 08 2026 06:05:10 GMT+0000 (UTC))
datadir: /root/.ethereum
modules: admin:1.0 clique:1.0 debug:1.0 engine:1.0 eth:1.0 miner:1.0 net:1.0 personal:1.0 rpc:1.0
txpool:1.0 web3:1.0
```

5.1 查询余额 (Task 3.a)

方法一：不进入容器，宿主机一条命令执行 (推荐)

在宿主机 `Labsetup` 目录下直接运行（把容器名换成你 `dockps` 看到的）：

```
# Account 0
docker exec as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 \
  geth attach --exec
'web3.fromWei(eth.getBalance("0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"), "ether")'

# Account 1
docker exec as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 \
  geth attach --exec
'web3.fromWei(eth.getBalance("0x2e2e3a61dac1A2056d9304F79C168cD16aAa88e9"), "ether")'

# Account 2
docker exec as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 \
  geth attach --exec
'web3.fromWei(eth.getBalance("0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"), "ether")'
```

每条命令会输出一个数字（单位 ETH），例如 `29.19`、`9999995.5`、`32` 等（随前面 Task 1/2 转账变化）。

方法二：进入容器后交互执行

```
# 1. 宿主机：进入容器
docker exec -it as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 /bin/bash

# 2. 容器内：打开 Geth 控制台
geth attach

# 3. 在 > 提示符下逐条输入（不要用 print，直接回车即可看到返回值）
web3.fromWei(eth.getBalance("0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"), "ether")
web3.fromWei(eth.getBalance("0x2e2e3a61dac1A2056d9304F79C168cD16aAa88e9"), "ether")
web3.fromWei(eth.getBalance("0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"), "ether")
```

实验结果（与 Python/MetaMask 对比）：

账户	Geth 余额	与 MetaMask 一致
Account 1	与 MetaMask 相同	✓
Account 2	与 MetaMask 相同	✓
Account 3	与 MetaMask 相同	✓

```
To exit, press ctrl-d or type exit
> web3.fromWei(eth.getBalance("0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9"), "ether")
7.199806159850889
> web3.fromWei(eth.getBalance("0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9"), "ether")
9999975.49987233268043
> web3.fromWei(eth.getBalance("0xCBf1e330F0abD5c1ac979CF2B2B874cfD4902E24"), "ether")
53
```

5.2 从节点本地账户发送（Task 3.b）

节点 `eth.accounts` 维护的是 **keystore 中的本地账户**，与 MetaMask 账户不同。

查看节点本地账户（宿主机一条命令）：

```
docker exec as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 \
  geth attach --exec 'eth.accounts'
# 示例输出: ["0x1be9288f9a7d2f809250f15c487e3a5a9cf71f4f",
"0xa403f63ad02a557d5ddcbd5f5af9a7627c591034"]

> eth.accounts
["0x1be9288f9a7d2f809250f15c487e3a5a9cf71f4f", "0xa403f63ad02a557d5ddcbd5f5af9a7627c591034"]
```

发送 0.5 ETH 到 Account 2（宿主机一条命令）：

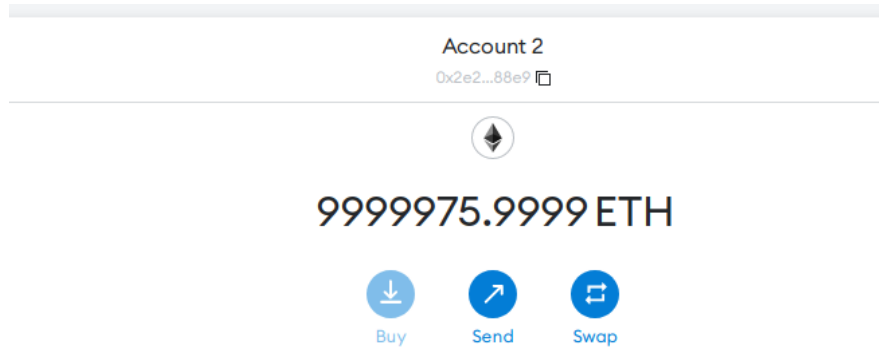
```
docker exec as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 \
  geth attach --exec 'personal.unlockAccount(eth.accounts[0], "admin", 300);
eth.sendTransaction({from: eth.accounts[0], to:
"0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9", value: web3.toWei(0.5, "ether")})'
```

或在容器内 `geth attach` 后逐条输入：

```
personal.unlockAccount(eth.accounts[0], "admin", 300)
eth.sendTransaction({
  from: eth.accounts[0],
  to: "0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9",
  value: web3.toWei(0.5, "ether")
})
// 返回 TxHash, 例如: "0x423595c8e7fd2bb10e31225815b11a56de5d9a18fc8f1cb14c1c682d675ba08f"
```

观察：节点本地账户密码统一为 `admin`（见 `/tmp/eth-password`），unlock 后 Geth 代为签名，无需用户提供私钥。

```
> personal.unlockAccount(eth.accounts[0], "admin", 300); eth.sendTransaction({from: eth.accounts[0], to: "0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9", value: web3.toWei(0.5, "ether")})
"0x3027c86d7497bbb6692ee3eef72e8f68bacdbbfac8d3b48f192b74e5c0ae36e0"
```



确实多了0.5ETH。

5.3 从 MetaMask 账户发送 (Task 3.c)

在容器内 `geth attach`，或宿主机执行：

```
docker exec as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71 \
  geth attach --exec 'eth.sendTransaction({from:
"0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9", to:
"0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24", value: web3.toWei(1, "ether")})'
```

交互式写法：

```
eth.sendTransaction({
  from: "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9",
  to: "0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24",
  value: web3.toWei(1, "ether")
});
```

结果：

```
> eth.sendTransaction({from: "0xF5406927254d2dA7F7c28A61191e3Ff1f2400fe9", to: "0xCBF1e330F0abD5c1ac979CF2B2B874cfD4902E24", value: web3.toWei(1, "ether")})
Error: unknown account
    at web3.js:6365:9(45)
    at send (web3.js:5099:62(34))
    at <eval>:1:20(15)
```

原因分析：

Geth 的 `eth.sendTransaction` 只能操作 **本节点 keystore 中存在的账户**。MetaMask 账户的私钥存储在浏览器扩展中，不在节点的 `/root/.ethereum/keystore/` 目录下，因此 Geth 无法解锁和签名。

方式	私钥位置	签名者
MetaMask	浏览器本地	MetaMask
Python raw tx	脚本/环境变量	eth_account 库
Geth console	节点 keystore	Geth 节点

6. Task 4: 添加 Full Node

6.1 配置流程

空白容器 `as150h-new_eth_node-10.150.0.74` 由 `blockchain-poa.py` 预创建:

```
# Create a new empty ethereum node for one of the tasks
# This node will use a pre-build image
newhost = emu.getLayer('Base').getAutonomousSystem(150).createHost('new_eth_node')
newhost.joinNetwork('net0')
```

Step 0 — 确认容器名称 (在宿主机 `emulator_20` 目录):

```
docker ps --format "{{.Names}}" | grep -E "new_eth|Ethereum-POA"
```

```
[06/08/26]seed@VM:~/.../emulator_20$ docker ps --format "{{.Names}}" | grep -E "new_eth|Ethereum-POA"
as154h-Ethereum-POA-08-Signer-10.154.0.71
as161h-Ethereum-POA-12-Signer-BootNode-10.161.0.71
as150h-new_eth_node-10.150.0.74
as160h-Ethereum-POA-11-10.160.0.72
as150h-Ethereum-POA-01-10.150.0.72
as161h-Ethereum-POA-13-10.161.0.72
as164h-Ethereum-POA-18-Signer-BootNode-10.164.0.71
as160h-Ethereum-POA-10-Signer-10.160.0.71
as152h-Ethereum-POA-04-Signer-10.152.0.71
as150h-Ethereum-POA-00-Signer-BootNode-10.150.0.71
as151h-Ethereum-POA-02-Signer-10.151.0.71
as162h-Ethereum-POA-15-BootNode-10.162.0.72
as153h-Ethereum-POA-06-Signer-BootNode-10.153.0.71
as164h-Ethereum-POA-19-10.164.0.72
as154h-Ethereum-POA-09-BootNode-10.154.0.72
as153h-Ethereum-POA-07-10.153.0.72
as162h-Ethereum-POA-14-Signer-10.162.0.71
as152h-Ethereum-POA-05-10.152.0.72
as163h-Ethereum-POA-17-10.163.0.72
as163h-Ethereum-POA-16-Signer-10.163.0.71
as151h-Ethereum-POA-03-BootNode-10.151.0.72
```

Step 1 — 从已有节点复制创世块, 并初始化新节点:

1. 停掉容器里的 geth

```
docker exec as150h-new_eth_node-10.150.0.74 pkill geth
```

2. 清空数据目录

```
docker exec as150h-new_eth_node-10.150.0.74 rm -rf /root/.ethereum/*
```

1a. 从已有节点 → 宿主机临时目录

```
docker cp as150h-Ethereum-POA-01-10.150.0.72:/tmp/eth-genesis.json /tmp/eth-genesis.json
```

1b. 从宿主机 → 新节点容器

```
docker cp /tmp/eth-genesis.json as150h-new_eth_node-10.150.0.74:/eth-genesis.json
```

1c. 在新节点容器内初始化区块链，这个已经搞定了，不用再运行。

```
docker exec as150h-new_eth_node-10.150.0.74 geth --datadir /root/.ethereum init /eth-genesis.json
```

```
[06/08/26]seed@VM:~/.../emulator_20$ docker exec as150h-new_eth_node-10.150.0.74 pkill geth
[06/08/26]seed@VM:~/.../emulator_20$ docker exec as150h-new_eth_node-10.150.0.74 rm -rf /root/.ethereum/*
[06/08/26]seed@VM:~/.../emulator_20$ docker cp as150h-Ethereum-POA-01-10.150.0.72:/tmp/eth-genesis.json /tmp/eth-genesis.json
[06/08/26]seed@VM:~/.../emulator_20$ docker cp /tmp/eth-genesis.json as150h-new_eth_node-10.150.0.74:/eth-genesis.json
[06/08/26]seed@VM:~/.../emulator_20$ docker exec as150h-new_eth_node-10.150.0.74 geth --datadir /root/.ethereum init /eth-genesis.json
INFO [06-08|06:59:36.499] Maximum peer count                      ETH=50 LES=0 total=50
INFO [06-08|06:59:36.517] Smartcard socket not found, disabling   err="stat /run/pcscd/pcscd.comm: no such file or directory"
INFO [06-08|06:59:36.528] Set global gas cap                      cap=50,000,000
INFO [06-08|06:59:36.685] Allocated cache and file handles        database=/root/.ethereum/geth/chaindata cache=16.00MiB handles=16
INFO [06-08|06:59:36.922] Opened ancient database                 database=/root/.ethereum/geth/chaindata/ancient/chain readonly=false
INFO [06-08|06:59:36.967] Successfully wrote genesis state        database=chaindata hash=466467..3d4a54
INFO [06-08|06:59:36.970] Allocated cache and file handles        database=/root/.ethereum/geth/lightchaindata cache=16.00MiB handles=16
INFO [06-08|06:59:38.753] Opened ancient database                 database=/root/.ethereum/geth/lightchaindata/ancient/chain readonly=false
INFO [06-08|06:59:39.626] Successfully wrote genesis state        database=lightchaindata hash=466467..3d4a54
```

Step 2 — 复制 bootnode 列表并启动 Geth:

2a. 复制 bootnode 列表到新节点

```
docker cp as150h-Ethereum-POA-01-10.150.0.72:/tmp/eth-node-urls /tmp/eth-node-urls
```

```
docker cp /tmp/eth-node-urls as150h-new_eth_node-10.150.0.74:/tmp/eth-node-urls
```

2b. 在新节点内创建启动脚本并后台运行

```
docker exec as150h-new_eth_node-10.150.0.74 bash -c 'cat > /start.sh << "EOF"
```

```
#!/bin/bash
```

```
geth --datadir /root/.ethereum --identity="NEW_NODE_01" --networkid=1337 \
```

```
--syncmode full --snapshot=False --verbosity=2 --port 30303 \
```

```
--bootnodes "$(cat /tmp/eth-node-urls)" --allow-insecure-unlock \
```

```
--http --http.addr 0.0.0.0 --http.corsdomain "*" \
```

```
--http.api web3,eth,debug,personal,net,cliq,engine,admin,txpool
```

```
EOF
```

```
chmod +x /start.sh'
```

```
docker exec -d as150h-new_eth_node-10.150.0.74 bash -c 'nohup /start.sh > /tmp/geth.log 2>&1 &'
```

2c. 等待几秒后验证 Geth 已启动

```
sleep 10
```

```
docker exec as150h-new_eth_node-10.150.0.74 ps aux | grep geth | grep -v grep
```

```
[06/08/26]seed@VM:~/.../emulator_20$ docker exec as150h-new_eth_node-10.150.0.74 ps aux | grep geth
| grep -v grep
root      237   3.7   1.7 1813276 71200 ?        S1   07:00   0:01 geth --datadir /root/.ethereum -
--identity=NEW_NODE_01 --networkid=1337 --syncmode full --snapshot=False --verbosity=2 --port 30303
--bootnodes enode://45fa322fb033fa8325aa81ce2cfa7aa55daa488f1a1f35de8d1b2c9d498a1424a58cb84c2b37ab4
9106d8dfe277ba6aafba79aaf13ea347ea7fa46fa38ba63a1@10.150.0.71:30301, enode://bceff9af2ef8534763bcd1
04885139deec237bbd35a438c1c2b849e9519a35b7f411d32ae28c10551e5a2aa2ab54b82cbb8cb915266dcd1b55ff8ff44
80ebe73@10.151.0.72:30301, enode://18bb8241d53b259f2351b72a8f41bed8869fe296ec303936b366f94a2c88d3af
0a81ad571f229f73b6eae786b6e110f72973896262617763c4b54adde681ce58@10.153.0.71:30301, enode://33b339b
027965dbef23db9d360027169887443a42f5f7a84fe92fa2b9a3355898cdf84bea27eb48c99ccaeaa27d628c07b5a645678
1c59bb5a2f4ff36fe3bb75@10.154.0.72:30301, enode://198119127c42325bfaf82e7471b6a9fc3b45ab1427865624a
7eaf0d7495cd1334fe7f2f40b978de293bbd19c4e10ae0e71e830eba76105a6192510f95d1369ca@10.161.0.71:30301,
enode://c2cc2b48f4918f6d5a59f671b9b17a859f5717ef36aced561783d9843f5570d546d62fdcf562e1cf1a24d7061e3
cc2a1739f572260027276b56925d5dbc65e38@10.162.0.72:30301, enode://602d41737655bb0e44ad39db376b8cb7ef
9ad4172c8f826a77efe81b009870481ec046190232f7b62e8871fbbfd911c6e2b03b4be38ca10a6d6f3445ad64c4eb@10.1
64.0.71:30301, --allow-insecure-unlock --http --http.addr 0.0.0.0 --http.corsdomain * --http.api we
b3,eth,debug,personal,net,clique,engine,admin,txpool
```

6.2 验证 Peers

// 进入 Geth 控制台

```
docker exec -it as150h-new_eth_node-10.150.0.74 geth attach
```

// 在 > 提示符下查 peers

```
admin.peers.length // 20
```

```
admin.peers.map(p => p.network.remoteAddress)
```

```
// ["10.150.0.71:30303", "10.151.0.71:30303", ...]
```

新节点通过 bootnode enode URL 发现网络中的其他节点，建立 P2P 连接并同步区块。

```
> admin.peers.length
20
> admin.peers.map(p => p.network.remoteAddress)
["10.162.0.71:30303", "10.152.0.72:30303", "10.164.0.71:30303", "10.163.0.72:30303", "10.154.0.71:3
0303", "10.150.0.71:30303", "10.164.0.72:30303", "10.161.0.72:30303", "10.152.0.71:30303", "10.153.
0.72:30303", "10.151.0.72:30303", "10.161.0.71:30303", "10.150.0.72:30303", "10.154.0.72:30303", "1
0.162.0.72:30303", "10.151.0.71:30303", "10.153.0.71:30303", "10.163.0.71:30303", "10.160.0.72:3030
3", "10.160.0.71:30303"]
```

6.3 创建 Account Z

```
personal.newAccount("admin")
```

```
> personal.newAccount("admin")
"0x2d3eab8238ca485a13de7ef033469b41febe922e"
```

6.4 经新节点发送交易 (Python)

目标： 修改 Task 2 的 Python 脚本，把 RPC 指向**新节点** 10.150.0.74，从 **Account 1** 向 **Account Z** 发送 5 ETH。

Step 1 — 确认新节点 RPC 可用

在宿主机 `Labsetup` 目录执行：

```
curl -s -X POST -H "Content-Type: application/json" --data
'{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}' http://10.150.0.74:8545
```

若返回 `"result":"0x..."` 说明新节点 HTTP RPC 正常。若连不上，先确认 Geth 在跑：

```
docker exec as150h-new_eth_node-10.150.0.74 ps aux | grep geth
```

```
● [06/08/26]seed@VM:~/.../Labsetup$ curl -s -X POST -H "Content-Type: application/json" --data
'{"jsonrpc":"2.0","method":"eth_blockNumber","params":[],"id":1}' http://10.150.0.74:8545
{"jsonrpc":"2.0","id":1,"result":"0x14"}
```

Step 2 — 使用 Task 4 专用脚本 (推荐)

已提供 `Files/web3_raw_tx_task4.py`：

```
#!/bin/env python3
"""Task 4: 通过新节点 RPC 向 Account Z 发送 5 ETH"""
from web3 import Web3
from eth_account import Account

# 新节点 IP (容器名 as150h-new_eth_node-10.150.0.74)
web3 = Web3(Web3.HTTPProvider('http://10.150.0.74:8545'))

# Account 1 私钥 (助记词派生的第二个账户)
key = '0x20aec3a7207fcda31bdef03001d9caf89179954879e595d9a190d6ac8204e498'
sender = Account.from_key(key)

# Account Z 地址 (必须与 6.3 personal.newAccount 返回的一致!)
recipient = Web3.toChecksumAddress('0x2d3eab8238ca485a13de7ef033469b41febe922e')

print("Connected:", web3.isConnected())
print("Sender (Account 1):", sender.address)
print("Recipient (Account Z):", recipient)
print("Account Z balance before:", web3.fromWei(web3.eth.get_balance(recipient), 'ether'),
      "ETH")

tx = {
    'chainId': 1337,
    'nonce': web3.eth.getTransactionCount(sender.address),
    'from': sender.address,
    'to': recipient,
    'value': web3.toWei("5", 'ether'),
    'gas': 200000,
    'maxFeePerGas': web3.toWei('4', 'gwei'),
```

```

    'maxPriorityFeePerGas': web3.toWei('3', 'gwei'),
    'data': ''
}

signed_tx = web3.eth.account.sign_transaction(tx, sender.key)
tx_hash = web3.eth.sendRawTransaction(signed_tx.rawTransaction)
print("Tx sent, waiting for receipt ...")
receipt = web3.eth.wait_for_transaction_receipt(tx_hash)

print("\n===== Transaction Receipt =====")
print("Status:      ", "Success" if receipt.status == 1 else "Failed")
print("Tx Hash:      ", receipt.transactionHash.hex())
print("Block Number: ", receipt.blockNumber)
print("From:          ", receipt['from'])
print("To:            ", receipt.to)
print("=====")
print("Account Z balance after:", web3.fromWei(web3.eth.get_balance(recipient), 'ether'),
      "ETH")

```

Step 3 — 运行脚本

```

cd ~/Desktop/Labsetup
python3 Files/web3_raw_tx_task4.py

```

成功输出:

```

• [06/08/26]seed@VM:~/.../Labsetup$ python3 Files/web3_raw_tx_task4.py
/usr/lib/python3/dist-packages/requests/__init__.py:89: RequestsDependencyWarning: urllib3 (2.2.3)
or chardet (3.0.4) doesn't match a supported version!
  warnings.warn("urllib3 ({}) or chardet ({}) doesn't match a supported "
Connected: True
Sender (Account 1): 0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9
Recipient (Account Z): 0x2D3eAB8238CA485a13de7ef033469b41FeBe922E
Account Z balance before: 0 ETH
Tx sent, waiting for receipt ...

===== Transaction Receipt =====
Status:      Success
Tx Hash:      0x3f534540d4e6ad9678d5fc88793e05d896d0773ac046f9ce1a742f4955772883
Block Number: 47
From:         0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9
To:           0x2D3eAB8238CA485a13de7ef033469b41FeBe922E
=====
Account Z balance after: 5 ETH

```

也可在 MetaMask 中查看 Account 1 余额减少、Account Z 需在新节点 `geth attach` 里用 `eth.getBalance(eth.accounts[0])` 查看。

说明： RPC 连哪个节点只是提交交易的入口；打包出块仍由 PoA 签名者完成，与连 `10.150.0.71` 还是 `10.150.0.74` 无关。

6.5 从 Account Z 发出交易 (Geth 控制台)

目标： 在新节点的 Geth 控制台里，用 Account Z（节点本地 keystore 账户）向 Account 1 转 **1 ETH**。

Step 1 — 进入新节点 Geth 控制台

```
docker exec -it as150h-new_eth_node-10.150.0.74 geth attach
```

Step 2 — 确认 Account Z 是本地第一个账户

```
eth.accounts
```

```
> eth.accounts  
["0x2d3eab8238ca485a13de7ef033469b41febe922e"]
```

Step 3 — 查看 Account Z 当前余额 (应有 6.4 转入的 5 ETH)

```
web3.fromWei(eth.getBalance(eth.accounts[0]), "ether")
```

```
> web3.fromWei(eth.getBalance(eth.accounts[0]), "ether")  
5
```

Step 4 — 解锁并发交易

在 `>` 提示符下逐条输入（密码是创建账户时设的 `"admin"`）：

```
personal.unlockAccount(eth.accounts[0], "admin", 300)
```

返回 `true` 后，再输入：

```
eth.sendTransaction({from: eth.accounts[0], to:  
  "0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9", value: web3.toWei(1, "ether")})
```

成功会返回交易哈希：

```
> personal.unlockAccount(eth.accounts[0], "admin", 300)  
true  
> eth.sendTransaction({from: eth.accounts[0], to: "0x2e2e3a61daC1A2056d9304F79C168cD16aAa88e9", value: web3.toWei(1, "ether")})  
"0x9756ac288b3af3a62dba088c44af67f0dedbbc8429532e08d0a9d6ef8d1a3a90"
```

Step 5 — 验证余额变化

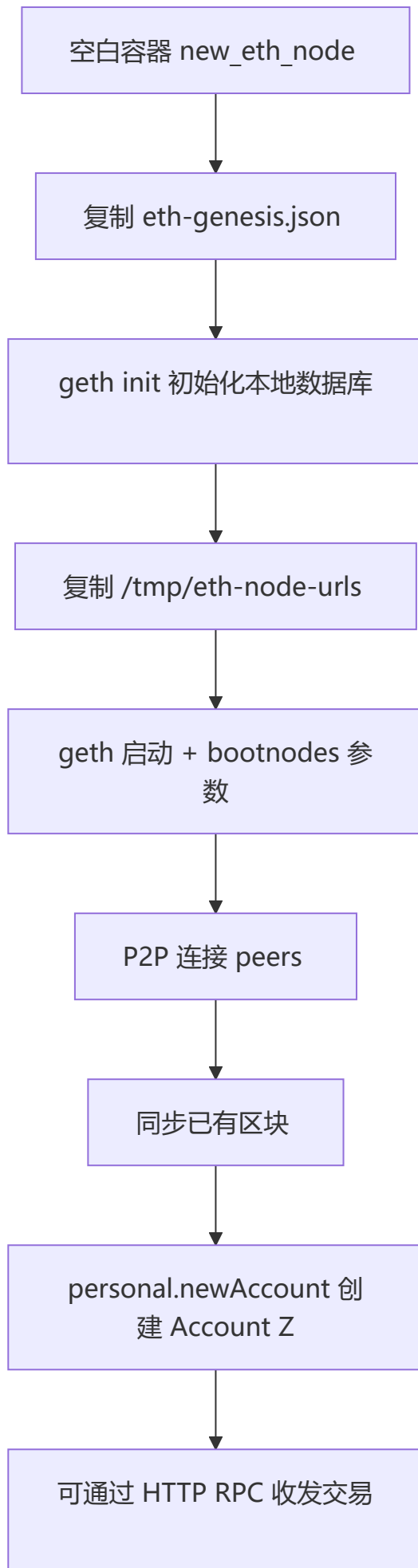
```
web3.fromWei(eth.getBalance(eth.accounts[0]), "ether")
```

Account Z 最终约 **3.999978 ETH** (5 ETH - 1 ETH - gas 费)。

也可在 MetaMask 的 Account 1 中看到余额增加约 1 ETH。

```
> web3.fromWei(eth.getBalance(eth.accounts[0]), "ether")  
3.999978986393848
```

6.6 加入网络流程图



7. 问题回答与观察

Q1：为什么实验使用 PoA 而不是 PoS？

PoA 由预授权节点（Signer）轮流签名出块，配置简单、出块稳定，适合在 Docker 仿真环境中快速搭建私有链。主网 PoS 需要质押、验证者选举和罚没机制，复杂度高，不适合本实验教学目标。

Q2：助记词（Mnemonic）的作用是什么？

助记词是 BIP-39 标准的 12/24 个英文单词，可确定性派生无限个账户的私钥。MetaMask "Forgot password" 功能依赖助记词恢复钱包——**密码只加密本地存储，私钥真正由助记词派生**。因此助记词必须离线妥善保管。

Q3：为什么 Geth 控制台无法从 MetaMask 账户发交易？

Geth `eth.sendTransaction` 要求 `from` 地址存在于节点 keystore。MetaMask 账户私钥不在节点上，报 `unknown account`。解决方案：使用 MetaMask 自行签名，或用 Python 构造 raw transaction。

Q4：连接不同节点 RPC 发送交易有区别吗？

无本质区别。JSON-RPC 节点只是入口，交易进入 TxPool 后由 PoA 签名者打包。本实验中通过 `10.150.0.71` 和 `10.150.0.74` 发送的交易均成功上链。

Q5：PoA 网络中 bootnode 的作用？

Bootnode 提供已知 enode 地址，帮助新节点发现网络中的其他 peer。新节点启动时通过 `--bootnodes` 参数连接这些节点，完成 P2P 拓扑构建和区块同步。

Q6：gas 费用如何影响余额？

简单 ETH 转账消耗约 21000 gas。Account 0 发送 11 ETH 后余额为 19.199932 ETH（而非 19.2 ETH），差额为 gas 费（约 0.000068 ETH）。Account Z 发送 1 ETH 后剩余 3.999978 ETH，同样扣除了 gas。

8. 实验结论

1. **MetaMask** 提供了友好的 GUI 层，通过 JSON-RPC 连接节点，在客户端完成密钥管理与签名。
2. **Python web3.py** 展示了底层交互：构造交易 → 本地签名 → 广播 raw tx → 等待 receipt，适合自动化和集成开发。
3. **Geth 控制台** 直接操作节点 keystore 账户，适合节点运维和调试，但无法代签外部钱包账户。
4. **新节点加入** 需 genesis 初始化 + bootnode 配置，验证了以太坊 P2P 网络的即插即用特性。

本实验完整走通了以太坊账户体系、交易生命周期、JSON-RPC 接口和节点组网流程，为后续智能合约实验打下基础。